



LABORATORY OF
IMAGE
SYNTHESIS AND
ANALYSIS



Rapport de stage technique (S7) - 2025/2026

Mise en place d'une application web avec Frappe/ERPNext pour la gestion des sujets de mémoire

LABORATOIRE D'ACCUEIL : Laboratory of Image Synthesis and Analysis (LISA) –
Université Libre de Bruxelles (ULB)

TUTEUR EN ENTREPRISE : DEBEIR Olivier - ERCEK Rudy

DATES : du 01/09/2025 au 29/12/2025

TUTEUR PÉDAGOGIQUE ESEO : SCHANG Daniel

BOUDLAL Nargice

15 décembre 2025

ENGAGEMENTS :

Je soussigné(e), Nargice BOUDLAL, étudiante à l'ESEO, atteste avoir pris connaissance du contenu du Règlement intérieur de l'École et de l'engagement de « non-plagiat ». Je déclare m'y conformer dans le cadre de la rédaction de ce document. Je déclare sur l'honneur que le contenu du présent mémoire est original et reflète mon travail personnel. J'atteste que les citations sont correctement signalées par des guillemets et que les sources de tous les emprunts ponctuels à d'autres auteur(e)s, textuels ou non textuels, sont indiquées. Le non-respect de cet engagement m'exposerait à des sanctions dont j'ai bien pris connaissance.

Fait à Bruxelles, Belgique, le 03/11/2025

DÉCLARATION RELATIVE À L'IA GÉNÉRATIVE :

*Durant la rédaction de ce rapport, j'ai utilisé une IA générative (ChatGPT) pour des tâches de **reformulation**, **structuration** et **relecture**. Après utilisation, j'ai relu et édité le contenu produit et **j'en assume l'entière responsabilité**.*

REMERCIEMENTS :

Je tiens tout d'abord à adresser mes sincères remerciements à mes tuteurs, **M. Olivier Debeir** et **M. Rudy Ercek**, pour leur accompagnement, leurs précieux conseils et leur suivi tout au long du stage. Je remercie tout particulièrement **M. Olivier Debeir**, qui m'a donné l'opportunité d'intégrer le laboratoire et de réaliser ce projet au sein du LISA.

Je remercie ensuite chaleureusement toute l'équipe du **Laboratory of Image Synthesis and Analysis (LISA)** pour son accueil bienveillant et son aide tout au long de cette expérience. Leur disponibilité, leur esprit d'équipe et leur ambiance de travail conviviale ont grandement contribué à la réussite de mon stage. Je remercie notamment **Arlette** et **Arthur** pour leur soutien, leurs échanges et leur disponibilité au quotidien.

Enfin, je remercie **Alexandre Borreda**, étudiant à **Polytech Lyon** et stagiaire au LISA en même temps que moi, pour les échanges, l'entraide et l'esprit de collaboration qui ont rendu cette expérience encore plus enrichissante.

SOMMAIRE :

ENGAGEMENTS :	1
DÉCLARATION RELATIVE À L'IA GÉNÉRATIVE :	1
REMERCIEMENTS :	2
FICHE DE SYNTHÈSE :	5
I. Abstract :	5
II. Résumé :	6
INTRODUCTION :	7
I. Contexte :	7
1. Présentation du LISA	7
2. Contexte du projet	8
II. Objectifs et impacts :	9
1. Objectifs SMART	9
2. Problématiques	10
DÉVELOPPEMENT :	10
I. Cahier des charges	10
1. Acteurs	10
2. Espace Étudiant – Student space	11
3. Espace Professeur – Professor space	11
4. Espace Administration – Administration space	11
II. Organisation :	12
III. Mise en oeuvre	12
1. ERPNext	12
2. Architecture technique et technologique	13
3. Problématiques & obstacles rencontrés	14
4. Configuration et Choix de l'Environnement de Développement	15
4.1 Première phase : Frappe Docker	15
4.2 Seconde phase : Environnement de Développement Local (Bench)	17
4.3 Stratégie de Déploiement et d'Installation de l'Application	17
5. Développement de l'application	18
5.1 Préparation de l'Outil Bench sur Linux	18
5.2 Création du Site et de l'Application	19
5.3 Fil conducteur	20
5.4 Modélisation dans le Desk Frappe	20
5.5 Développement Full Code dans VS Code	21

5.6 Le Moteur de l'Interaction (Le Flux JS/API/Python)	22
5.7 Implémentation des Fonctionnalités Clés	23
5.7.1. L'Administration	23
5.7.2. Le Flux Étudiant	25
5.7.3. Le Flux Professeur	29
5.7.4. Gestion des années académiques – utils.py	33
5.7.5. Logique serveur api.py et hooks.py	34
5.7.6. GitLab	35
6. Mise en place de l'envoi des notifications par e-mail	36
6.1. Configuration du compte e-mail	37
6.2. Intégration dans l'application	38
7. Déploiement Docker	38
7.1. Installation de l'environnement Frappe Docker	38
7.2. Création du site Frappe thesis.lisa.ulb.ac.be	39
7.3. Déploiement de l'application thesis_application depuis GitLab	40
7.4. Redémarrage du backend après déploiement	41
IMPACT DU PROJET	41
I. Impact écologique	41
II. Impact social	42
CONCLUSION	42
ANNEXES	44
Annexe A — Installation & Déploiement	44
A.1 – Script d'installation de l'environnement local (Bench)	44
A.2 – Structure type d'une application Frappe	45
A.3 – Arborescence du projet thesis_application	45
A.4 – Script de déploiement Docker	46
Annexe B — Notification par mail	48
Annexe C — Principales ressources de documentation utilisées	49
Annexe D — Table des figures et illustrations utilisées dans le rapport	50
Annexe E — Glossaire des définitions et acronymes utilisés	50

FICHE DE SYNTHÈSE :

Sujet :

Conception, développement et déploiement d'un module de gestion des mémoires de fin d'études sous **ERPNext/Frappe** (remplacement d'un dispositif **LimeSurvey + scripts Python** devenu incompatible avec les mises à jour et difficile à maintenir).

Organisation d'accueil :

Laboratory of Image Synthesis and Analysis (**LISA**) — Université Libre de Bruxelles (**ULB**), Bruxelles, Belgique

Période : 01/09/2025 – 29/12/2025

I. Abstract :

This internship focused on the development and deployment of a thesis-management web application module using **ERPNext/Frappe** (Python, MariaDB, JavaScript) and **Docker**. The project took place at the **Laboratory of Image Synthesis and Analysis (LISA)**, **Université Libre de Bruxelles (ULB)**, in **Brussels, Belgium**, from **01/09/2025 to 29/12/2025**, under the supervision of **Olivier Debeir** and **Rudy Ercek**. The starting point was an existing process relying on LimeSurvey combined with custom Python scripts, which had become fragile after platform updates and difficult to maintain, resulting in limited traceability and excessive informal email exchanges. The key question was therefore how to replace this outdated workflow with an **open-source ERP-based** solution that is robust, evolvable, and able to support a unified process for students from both **ULB and VUB**. The objectives were to deliver an end-to-end workflow covering topic publication, student applications, supervisor decisions with traceability, student follow-up, final PDF submission, and an administration overview including exports for a selected academic year. To achieve this, I first modeled the business data in Frappe by defining DocTypes and relationships and by implementing role-based access control

Nargice BOUDLAL - Rapport de stage - ESEO

(Student/Professor/Administration). I then built dedicated web pages (Jinja2/HTML/CSS/JavaScript) backed by whitelisted Python APIs, and automated notifications through Frappe hooks on Thesis Application events to email the professor upon a new application and the student upon a decision. Development was conducted locally with **bench** for rapid iteration and versioned in GitLab, before being deployed reproducibly using **Frappe Docker** on a dedicated site (thesis.lisa.ulb.ac.be). The resulting module provides **three role-specific portals**, automatic role-based redirection after login, academic-year filtering, topic visibility management, ULB/VUB student identification, and a complete **CSV export of all applications** (Approved/Reject/On Hold) for a selected academic year. These outcomes demonstrate that ERPNext/Frappe can serve as a maintainable foundation for a custom academic workflow when data modeling, permissions, and event-driven automation are carefully designed; while the implementation targets the LISA/ULB context, the approach is generalizable to other administrative processes requiring traceability and strict role separation.

II. Résumé :

Ce stage a porté sur la conception, le développement et le déploiement d'un module web de gestion des sujets et candidatures de mémoire basé sur **ERPNext/Frappe** (Python, MariaDB, JavaScript) et **Docker**. Il s'est déroulé au **Laboratory of Image Synthesis and Analysis (LISA), Université Libre de Bruxelles (ULB), à Bruxelles (Belgique)**, du **01/09/2025 au 29/12/2025**, sous l'encadrement de **Olivier Debeir** et **Rudy Ercek**. Le point de départ était un dispositif existant reposant sur LimeSurvey et des scripts Python, devenu fragile après mise à jour et difficile à maintenir, ce qui limitait la traçabilité des décisions et multipliait les échanges informels par e-mail. La problématique centrale était donc de remplacer ce système obsolète par une solution **ERP open source** à la fois robuste, évolutive et capable de gérer un processus unique pour des étudiants **ULB et VUB**. Les objectifs étaient de mettre en place un flux complet : publication des sujets, candidatures, décisions des professeurs avec traçabilité, suivi côté étudiant, dépôt du PDF final, et supervision par l'administration avec export sur une année académique donnée. Pour y parvenir, j'ai d'abord modélisé les données dans Frappe en définissant les DocTypes, leurs relations, ainsi que les rôles et permissions (Student/Professor/Administration). J'ai ensuite développé des pages web dédiées (Jinja2/HTML/CSS/JavaScript) reliées à des API Python accessibles via

@frappe.whitelist, et automatisé les notifications via des hooks Frappe sur les événements du DocType Thesis Application afin d'envoyer un e-mail au professeur lors d'une candidature, puis à l'étudiant lors d'une décision. Le développement a été réalisé en local avec **bench** (itérations rapides) et versionné sur GitLab, puis déployé de façon reproductible avec **Frappe Docker** sur un site dédié (thesis.lisa.ulb.ac.be). Le module obtenu fournit **trois espaces** par rôle, la redirection automatique après connexion, le filtrage par année académique, la gestion de visibilité des sujets, l'identification ULB/VUB, et un **export CSV complet** de toutes les candidatures (Approved/Reject/On Hold) pour l'année sélectionnée. Ces résultats montrent qu'ERPNext/Frappe constitue un socle maintenable pour un flux métier académique dès lors que la modélisation des données, les permissions et l'automatisation par événements sont correctement structurées ; bien que conçu pour le contexte LISA/ULB, le principe est transposable à d'autres processus administratifs nécessitant traçabilité et séparation stricte des rôles.

INTRODUCTION :

I. Contexte :

1. Présentation du LISA

Le laboratoire LISA, **Laboratory of Image Synthesis and Analysis**, est une unité de recherche rattachée à l'**Université Libre de Bruxelles (ULB)**, qui est l'une des principales universités francophones de Belgique. Il est localisé sur le **campus du Solbosch** à Bruxelles, au 50 avenue Franklin Roosevelt, dans un environnement académique et de recherche dynamique.

Deux pôles complémentaires :

1. **Analyse d'images par Deep Learning** (*mon unité*) : principalement **imagerie médicale** (segmentation, classification, détection, registration), avec extensions à l'**imagerie satellite, infrarouge** etc...
2. **Réalité virtuelle / Vision par ordinateur** (*localisée dans un autre bâtiment*) : environnements immersifs et dispositifs de **computer vision**.

Positionnement : selon l’encadrement, le **LISA** est le **seul laboratoire de l’ULB** spécifiquement dédié à l’**analyse d’images** et se positionne comme le **plus spécialisé** du campus sur ce thème, notamment grâce à sa maîtrise des approches **deep learning**.

Mon périmètre : j’interviens dans le **pôle Analyse d’images**

Concurrence :

IRIDIA : laboratoire d’**intelligence artificielle** de l’ULB axé sur l’**intelligence en essaim**, les **métaheuristiques** pour l’optimisation (combinatoire et continue), l’étude **fondamentale** de **réseaux biologiques** et des applications de **business intelligence**. IRIDIA est notamment connu pour la **métaheuristique Ant Colony Optimization**, des contributions en **stochastic local search** et le **contrôle de robots en essaim**.

Complémentarité : IRIDIA couvre des **fondamentaux IA** et de l’**optimisation** à large spectre ; le LISA se concentre sur l’**analyse/synthèse d’images** et la **vision** applicative. Les champs ne sont **pas concurrents au sens strict** mais **complémentaires** ; le LISA occupe la niche **image/vision** de l’ULB.

2. Contexte du projet

Durant mon stage, on m’a confié la refonte de l’application qui gère la publication des sujets de mémoire et les inscriptions des étudiants, initialement développée par le LISA pour la faculté. L’outil historique repose sur un assemblage de LimeSurvey et de scripts Python, devenu incompatible avec les mises à jour de la plateforme et difficile à maintenir.

La contrainte principale était de repartir sur un ERP robuste et open source comme socle technique, tout en prenant en compte le contexte particulier de Bruxelles : l’Université libre de Bruxelles (ULB, francophone) et la Vrije Universiteit Brussel (VUB, néerlandophone) proposent des masters mutualisés, et la nouvelle application devait donc permettre aux étudiants des deux universités de s’inscrire via un dispositif unique.

Pour répondre à ces contraintes et faciliter le travail des professeurs (publication de leurs sujets, suivi des candidatures, visibilité sur qui travaille sur quoi et avec quels encadrants),

j'ai proposé d'utiliser ERPNext comme base de la solution. Ce choix a été discuté puis validé avec mes encadrants.

II. Objectifs et impacts :

1. Objectifs SMART

S : Spécifique

L'objectif est de développer un module ERPNext/Frappe dédié et fonctionnel, déployé dans un environnement Docker, pour numériser l'intégralité du cycle de gestion des mémoires. Cela inclut la mise en place de trois portails dédiés (Étudiant, Professeur, Administration) après une analyse préalable du processus actuel et l'identification précise des besoins fonctionnels et techniques.

M : Mesurable

Le succès sera atteint si : tous les sujets actifs sont visibles sur le portail Étudiant ; le professeur peut traiter les candidatures avec succès (visibilité du statut côté étudiant) ; l'administration peut visualiser 100% des données et générer un export CSV complet ; le module est déployé et testé dans l'environnement Docker ; et une évaluation formelle des bénéfices et limites de la solution est réalisée.

A : Atteignable

La faisabilité est assurée par l'utilisation des outils et technologies existantes (ERPNext/Frappe), par la connexion des rôles et des données déjà en place, et par l'intégration du module dans un environnement de déploiement standardisé (Docker).

R : Réaliste

Le projet est réaliste car il répond à un besoin opérationnel fondamental de centralisation et de réduction des échanges d'e-mails dispersés. Il est soutenu par une démarche complète incluant l'analyse des processus (pour garantir l'adéquation fonctionnelle) et une évaluation des impacts écologiques (pour une approche holistique).

T : Temporel

L'analyse, la conception, le développement, le déploiement et l'évaluation finale du

module doivent être complétés, et l'ensemble des trois portails doit être pleinement fonctionnel, pour la date butoir du 29/12/2025.

2. Problématiques

Dans ce projet, on m'a imposé l'utilisation d'un ERP open source, donc gratuit, et suffisamment robuste, car l'établissement n'a ni équipe dédiée ni vraiment de temps pour maintenir un système complexe au quotidien. L'ancien outil (LimeSurvey) de gestion des sujets de mémoire avait justement cessé de fonctionner après une mise à jour et n'était plus maintenable. Je devais donc proposer une nouvelle solution qui permette de publier les sujets, de gérer les inscriptions des étudiants et le suivi par les enseignants, tout en reposant sur un socle technique stable, documenté et exploitable sur le long terme, sans intervention permanente d'un spécialiste.

Dans ce contexte, la question centrale de mon projet est la suivante :

Comment remplacer un ancien système de gestion des sujets de mémoire devenu inutilisable après mise à jour, par une solution basée sur un ERP, à la fois robuste, évolutive et suffisamment maîtrisée pour être maintenue dans la durée ?

DÉVELOPPEMENT :

I. Cahier des charges

L'outil doit permettre :

- la gestion des inscriptions/publication de sujets de mémoire
- de **réduire les échanges informels par e-mail** (perte d'informations, doublons, oubli de candidats) ;
- de **garantir la traçabilité** des candidatures et des décisions ;
- de **faciliter le travail des acteurs** (étudiants, promoteurs, responsables, administrateurs) en leur offrant des interfaces adaptées à leur rôle et ergonomique.

1. Acteurs

- **Étudiant** – espace *Student space*
- **Professeur / Promoteur** – espace *Professor space*
- **Administration** – espace *Administration space*

2. Espace Étudiant – *Student space*

Fonctionnalités attendues :

1. **Authentification**
 - Accès réservé aux utilisateurs connectés avec le rôle Étudiant.
2. **Tableau des sujets – “Thesis topics publication board”**
 - Affichage d’un tableau des sujets de mémoire :
 - **Title**
 - **Department**
 - **Supervisor** (email)

3. Espace Professeur – *Professor space*

Fonctionnalités attendues :

1. **Authentification**
 - Accès réservé aux utilisateurs ayant le rôle Professeur.
2. **Tableau des sujets**
 - Même tableau que côté étudiant (Title, Department, Supervisor, View details),
 - Affiche les sujets du professeur (et éventuellement tous les sujets dont il est responsable).

4. Espace Administration – *Administration space*

Fonctionnalités attendues :

1. **Authentification + contrôle de rôle**
 - Accès réservé aux utilisateurs avec le rôle **Administration**.
 - Redirection vers **/administration** à la connexion.

2. Tableau global des sujets

- Vue centralisée de **tous les sujets** de mémoire :
 - N°, Title, Department, Supervisor.
 - Colonne “**Visibility**” avec badge (ex. *visible* / *hidden*).
 - Bouton “**View details**”.
- L’administration peut consulter la **visibilité** de chaque sujet.

3. Export CSV

- Bouton “**Export CSV**” : CSV de toutes les candidatures de l’année sélectionnée (Approved/Reject/On Hold)

II. Organisation :

Mon approche a été séquencée en phases distinctes pour garantir une progression efficace. J’ai débuté par une phase de familiarisation intensive, qui a englobé l’analyse des besoins et la prise en main des technologies clés : l’écosystème ERPNext, le framework Frappe, ses API et les concepts de Docker.

Une fois cette fondation établie, je suis passé à la **phase de développement**. J’ai mis en place mon environnement local avec l’outil **Bench** et j’ai développé ma web application en utilisant **Visual Studio Code**. Enfin, la dernière étape fut dédiée à la **finalisation et au déploiement** : j’ai utilisé la conteneurisation via **Docker** pour migrer l’application du local vers un environnement reproductible, avant de procéder aux tests, à la documentation et à la mise en production.

III. Mise en oeuvre

1. ERPNext

L’utilisation d’un ERP était une contrainte imposée pour la réalisation de ce projet. Un **ERP** (*Enterprise Resource Planning*), ou progiciel de gestion intégré (PGI), est un logiciel qui permet de regrouper plusieurs fonctions d’une organisation (gestion des utilisateurs, projets, etc.) dans un même système. L’idée principale est d’avoir une base de données unique et des processus centralisés. Dans le cadre de ce projet, l’ERP est utilisé comme fondation technique pour gérer les différents rôles (étudiant, professeur, administration), les données liées aux sujets, les candidatures et les workflows associés.

Après avoir effectué une phase de recherche approfondie et une analyse comparative de différentes solutions ERP Open Source, mon choix s'est porté sur **ERPNext**, basé sur le framework **Frappe**. Cet ERP répondait aux contraintes de robustesse et est Open Source. Ce choix a été déterminé par trois caractéristiques techniques :

1. **Technologie (Python)** : L'utilisation de Python 3 pour le framework Frappe assure un développement fiable et une lisibilité du code essentielle pour la maintenance à long terme par une équipe non-spécialisée.
2. **Solution Complète et Sûre** : Frappe fournit un socle technique standardisé, intégrant nativement la gestion des *doctype*s, des rôles, des utilisateurs, l'authentification et les permissions.
3. **Open Source et Pérennité** : ERPNext/Frappe est une solution 100% Open Source, répondant à la contrainte budgétaire. Sa forte activité communautaire garantit la durabilité du projet et simplifie les mises à jour futures.

2. Architecture technique et technologique

L'architecture d'ERPNext est un facteur clé de sa flexibilité et de sa modernité. Elle repose intégralement sur le **Frappe Framework**, un environnement de développement *full-stack* basé sur un modèle orienté métadonnées.

Le **Frappe Framework** fournit la structure technique unifiée à l'ensemble des applications du système.

- **Pile Technologique (Stack) :**
 - **Backend & Logique Métier** : Le développement est majoritairement réalisé en Python, exploitant sa robustesse pour les processus d'entreprise complexes. Le système utilise une base de données MariaDB pour la persistance des données, comme illustré dans la Figure 1. Des outils comme Redis sont utilisés pour le caching et les files d'attente.
 - **Frontend & Expérience Utilisateur** : Frappe fournit le Desk pour l'administration et la configuration technique. Les interfaces destinées aux utilisateurs finaux sont développées via des pages web (www/, templates Jinja2/HTML/CSS + JavaScript) qui appellent le backend au moyen des

API Frappe (@frappe.whitelist).

- **Modèle de Données et Flexibilité** : Le concept central est le **DocType** (Type de Document). Chaque entité métier (Client, Facture, Ordre de Fabrication) est un **DocType**, géré par les métadonnées plutôt que par une programmation SQL rigide. Cette approche permet une **personnalisation profonde** et une extension rapide des modèles de données sans nécessiter de refonte structurelle majeure.

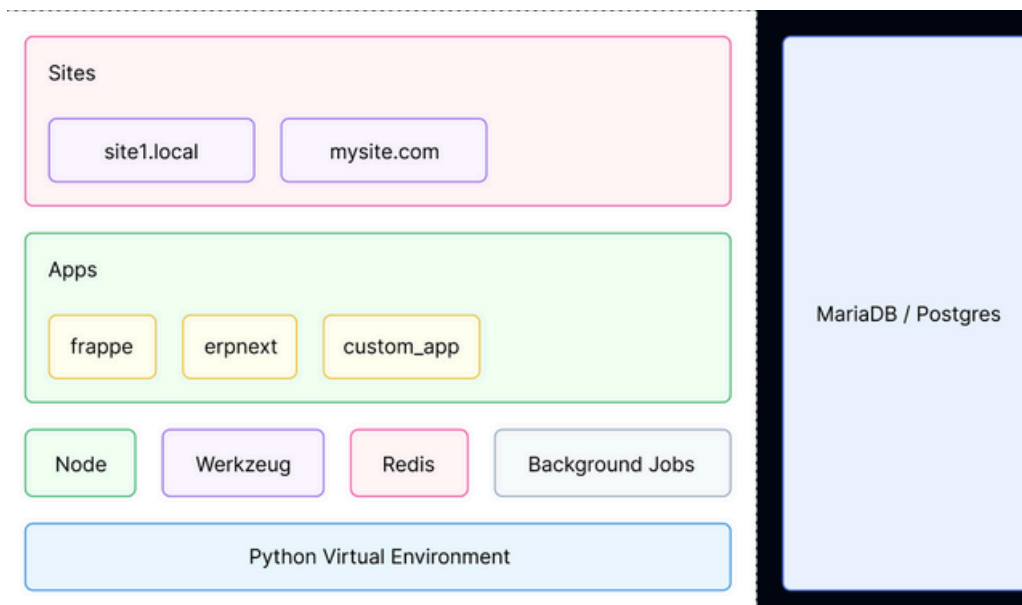


Figure 1 - Frappe Framework architecture

3. Problématiques & obstacles rencontrés

Le premier obstacle a été d'ordre architectural : j'ai initialement développé mon application web directement via l'interface du "Site" ERPNext, sans comprendre immédiatement que Frappe fonctionnait sur un principe d'**applications (Apps)** autonomes qui doivent être explicitement **installées** sur un **Site** Frappe.

Mon application de test, bien que fonctionnelle en développement, n'était pas reconnue ou déployable dans la structure *Apps* du Frappe Framework. Ce manque de clarté, suivi d'une

mauvaise manipulation du système de conteneurs ([docker compose down](#)), a entraîné la suppression accidentelle de l'environnement de travail et des configurations initiales.

4. Configuration et Choix de l'Environnement de Développement

Le développement d'applications sur la plateforme Frappe/ERPNext nécessite une compréhension approfondie de sa structure modulaire. Mon approche s'est déroulée en deux phases distinctes, suite à des défis rencontrés lors de l'initialisation de l'environnement de travail.

4.1 Première phase : Frappe Docker

Initialement, j'ai opté pour une approche moderne en utilisant l'environnement conteneurisé **Frappe Docker** ([frappe/frappe_docker](#)). Cette méthode permettait de démarrer rapidement l'intégralité du *stack* (Frappe, ERPNext, Base de Données, Redis) de manière isolée et standardisée.

Mais avant ça j'ai dû me familiariser avec cet outil et ses concepts :

1) Conteneurisation Docker :

La **conteneurisation** est une méthode d'encapsulation logicielle qui permet de livrer une application avec tout ce dont elle a besoin (code, dépendances, configurations) dans un **conteneur** isolé. Contrairement aux machines virtuelles, les conteneurs partagent le noyau du système d'exploitation hôte, ce qui les rend extrêmement **légers** et **rapides** à démarrer.

Le principal avantage est la **portabilité** : le conteneur assure un environnement d'exécution **standardisé et isolé**, garantissant que l'application se comporte de manière identique, quel que soit l'environnement (développement, test, production).

2) Docker Compose : Orchestration Multi-Conteneurs :

Une application d'entreprise comme ERPNext est une collection de services (base de données, application web, cache). **Docker Compose** est l'outil conçu pour gérer cet ensemble de composants.

Il utilise un fichier de configuration unique ([docker-compose.yml](#)) pour :

1. **Définir les Services** : Spécifier les conteneurs nécessaires (ex: [app](#), [db](#), [redis](#)).
2. **Réseautage** : Définir la manière dont ces conteneurs communiquent entre eux.
3. **Persistance des Données** : Configurer les **Volumes** pour que les données critiques survivent au redémarrage ou à la mise à jour des conteneurs.

En utilisant une seule commande, Docker Compose orchestre et lance l'intégralité de l'architecture applicative multi-conteneurs. C'est l'outil qui structure l'environnement **Frappe Docker**.

Dans cette configuration, l'application ERPNext s'exécute à l'intérieur d'un ou plusieurs conteneurs, le code source étant souvent monté en tant que **volume Docker** pour permettre le développement. L'exécution repose sur plusieurs conteneurs orchestrés par Docker Compose :

- Le conteneur **backend** héberge le Frappe Framework, qui sert l'application.
- Le conteneur **database** (MariaDB) stocke les données des sites créés.
- Chaque instance de travail (nommée "Site") est créée et configurée pour pointer vers cette base de données et est accessible via l'interface web (Desk).

Au début, j'ai commencé à construire l'application directement via le Desk de Frappe/ERPNext. Cette approche m'a permis de produire un premier brouillon et de comprendre le fonctionnement global de l'outil (création/paramétrage, premiers tests, etc.).

Ensuite, j'ai voulu passer à une approche plus structurée avec un Dev Container (environnement conteneurisé). Cependant, la mise en place s'est révélée trop complexe : j'ai rencontré plusieurs blocages et je n'ai pas réussi à obtenir un environnement stable pour continuer le développement efficacement.

En testant différentes manipulations pour débloquer la situation, j'ai également effectué une mauvaise commande (notamment un **docker compose down**), ce qui a interrompu mon environnement de travail et m'a fait perdre du temps.

Finalement, pour avancer plus rapidement sur le développement, j'ai choisi de basculer vers un environnement local avec frappe-bench, plus simple à maîtriser pendant la phase de développement.

4.2 Seconde phase : Environnement de Développement Local (Bench)

Face aux difficultés d'initialisation et de gestion des dépendances dans un environnement Docker complexe, j'ai pivoté vers l'environnement de développement local géré par **Bench** pour des raisons d'efficacité et de rapidité d'itération.

Le Rôle de Bench en Développement Local

L'utilisation de **Bench** en environnement local (sur la machine hôte) a permis de maîtriser l'intégralité du cycle de développement. Bench a été utilisé pour :

1. **Initialiser l'environnement Frappe** : Créer le dossier *bench*, installer les dépendances Python nécessaires, et configurer la base de données locale.
2. **Créer les Applications** : Utiliser les commandes **bench new-app** pour structurer correctement l'application selon les conventions du Frappe Framework.
3. **Intégration et Debugging** : Développer le code source de l'application en utilisant l'environnement **VS Code**, en parallèle avec la base de données et les dépendances gérées localement. Bench assure l'exécution des serveurs de développement et la supervision des changements de code.

4.3 Stratégie de Déploiement et d'Installation de l'Application

La stratégie de déploiement final s'appuie sur une chaîne de valeur claire pour la mise en production :

1. **Gestion des Versions (GitLab) :** Une fois le développement local terminé, j'ai versionné l'intégralité du **dossier d'application** et l'ai centralisé sur un dépôt **GitLab**.
2. **Déploiement et Installation (Frappe Docker) :** L'environnement de production utilise **Frappe Docker** pour son exécution sécurisée. Mon processus de déploiement suit les étapes suivantes :
 - a. **Création du Site :** Je commence par créer un nouveau Site Frappe dans l'environnement conteneurisé destiné à accueillir l'application.
 - b. **Récupération du Code Source :** Le processus de déploiement est configuré pour **cloner le dossier d'application** (mon module) directement depuis **GitLab** vers le répertoire des applications du conteneur Frappe.
 - c. **Installation Finale :** Une fois le code source cloné dans le conteneur, j'exécute la commande Bench nécessaire pour **installer formellement l'application sur le Site** créé.

Cette méthode garantit que mon code source, une fois validé en local, est déployé de manière standardisée et reproductible, permettant l'accès aux phases de tests finaux et la mise en production.

5. Développement de l'application

Ma première phase de travail a consisté à installer et configurer un environnement de développement local opérationnel, essentiel pour la suite du projet.

5.1 Préparation de l'Outil Bench sur Linux

J'ai d'abord configuré mon environnement logiciel en installant les prérequis : **Python 3.10**, **Node.js** avec **npm**, ainsi que les services **Redis** et **MariaDB**. J'ai systématiquement créé un environnement virtuel (`python3 -m venv env`) pour isoler les dépendances.

Ensuite, j'ai installé l'outil de gestion central de Frappe, bench, via **pip install frappe-bench**, puis j'ai initialisé mon dossier de travail : **bench init mon-bench**. Ce dossier contient toutes les applications (apps/) et les configurations des sites (sites/).

Le script complet d'installation de l'environnement local (Bench) est fourni en Annexe A.I.

5.2 Création du Site et de l'Application

J'ai créé mon site local, accessible via le nom **thesis.lisa.localhost**, où j'ai renseigné les mots de passe requis. Ce site utilise une base de données **MariaDB** sécurisée.

Mon application spécifique, **thesis_application**, est le pilier de mon projet, car elle repose directement sur le Frappe Framework pour fournir l'intégralité des structures nécessaires. Elle a été installée sur le site via les commandes suivantes :

```
bench new-app thesis_application
```

```
bench --site thesis.lisa.localhost install-app thesis_application
```

L'environnement de travail est ainsi accessible via l'URL <http://thesis.lisa.localhost:8000/> après avoir lancé la commande **bench start**.

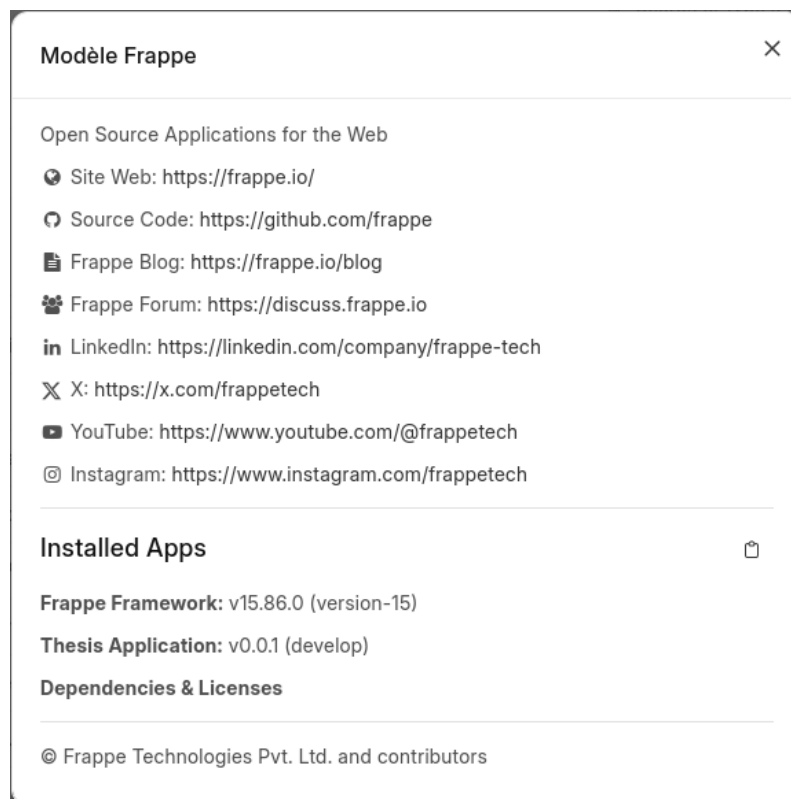


Figure 2 - Application “thesis_application” installé

5.3 Fil conducteur

Le fil conducteur du développement est de mettre en place un processus de candidature de bout en bout : l'étudiant postule, le professeur est automatiquement notifié et prend une décision (avec éventuellement un commentaire), l'étudiant reçoit une réponse et peut suivre le statut dans l'interface, tandis que l'administration dispose d'une vue globale et d'outils d'export pour le suivi.

5.4 Modélisation dans le Desk Frappe

En me connectant **en tant qu'Administrator** (profil administrateur) à partir du site que j'ai créé, j'accède au **Desk Frappe** afin de configurer la base du projet. Dans le contexte réel de l'application, ce profil correspond à la personne en charge du **service technique** (configuration, maintenance et gestion des paramètres).

Je commence donc **par modéliser les données** : je définis les **Doctypes** (entités métier), leurs **champs** (attributs) et les relations entre eux. Cette étape me permet de structurer correctement les informations manipulées par l'application (sujets, candidatures, utilisateurs, années académiques, etc.) et d'assurer une base cohérente pour l'ensemble des fonctionnalités.

En parallèle, **je définis les rôles et les droits d'accès** (étudiant, professeur, administration) afin de contrôler qui peut voir ou modifier quelles données, et d'adapter ensuite le comportement de l'application aux besoins de chaque profil.

Enfin, **j'utilise le Desk Frappe** (interface web interne) **uniquement pour ces aspects structurels et de configuration** : création/édition des Doctypes et champs, paramétrage des rôles et des permissions, et réglages généraux. **Je n'y développe aucune interface utilisateur** : les pages destinées aux utilisateurs finaux sont implémentées séparément via les routes web et les templates de l'application.

- **Création des DocTypes** : C'est dans le Desk que j'ai défini la structure complète de mes données (tables MariaDB). Les entités que j'ai modélisées sont : **Thesis Application, Thesis Topic, Academic session, Student Profile, Professor Profile, et Department**.

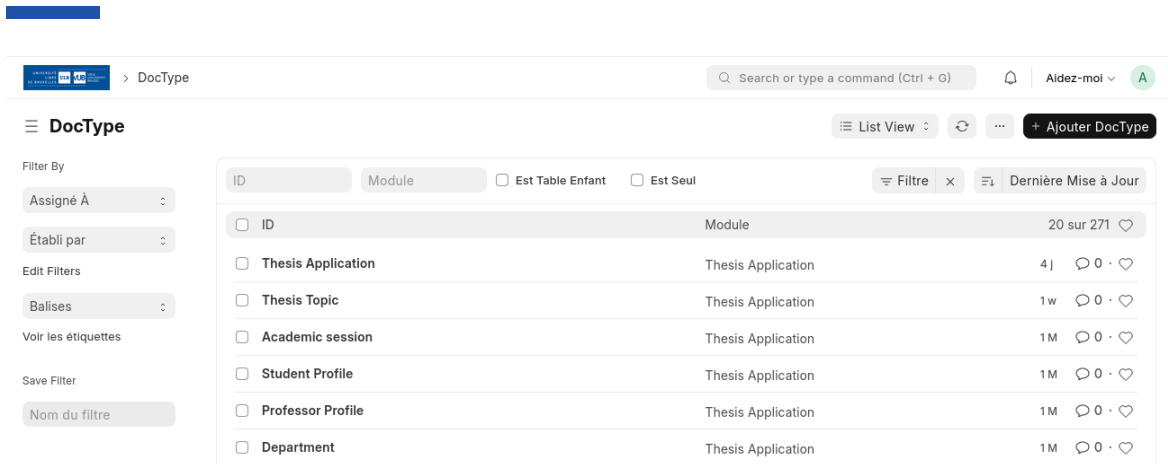


Figure 3 - Interface de gestion des doctypes

- **Rôles et Permissions** : J'ai créé les rôles utilisateurs clés, visibles dans le Desk : **Administration, Professor et Student**. J'ai géré, via le *Role Permission Manager*, les droits d'accès pour chaque rôle, assurant ainsi la sécurité et la confidentialité des données entre les différents utilisateurs.

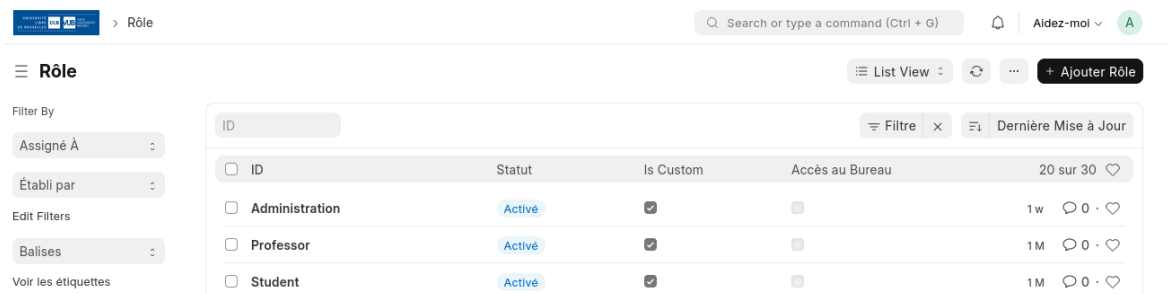


Figure 4 - Interface de gestion des rôles

5.5 Développement Full Code dans VS Code

Toute la logique métier et l'interface front-end sont gérées depuis **Visual Studio Code**, auquel j'accède directement via la commande dans le terminal : **code apps/thesis_application**.

Il est crucial de noter que la commande **bench new-app thesis_application** a généré automatiquement une structure de dossiers standardisée. C'est au sein de cette

arborescence pré-établie, incluant les dossiers templates/pages/ pour mes interfaces web et le fichier hooks.py pour la configuration globale, que j'ai pu insérer mon propre code Python et mes pages web.

La structure typique d'une application Frappe est détaillée en Annexe A.3 (Structure d'une Application).

L'architecture est basée sur le stack Frappe : **Python (Backend)**, **Jinja2 (Rendu Serveur)**, et **HTML/CSS/JavaScript (Frontend)**. Mon environnement de développement est lancé avec bench start.

5.6 Le Moteur de l'Interaction (Le Flux JS/API/Python)

L'application est dynamique car l'information voyage constamment entre le navigateur et la base de données. J'ai utilisé le modèle standard de Frappe, qui est comme une chaîne de montage :

1. **Côté Utilisateur (JavaScript)** : Quand je clique sur un bouton (par exemple, "Exporter CSV" ou "Upload PDF"), le **JavaScript (Frontend)** prend l'initiative.
2. **Lien vers le Serveur (API)** : Le JavaScript envoie une requête via **HTTP** au serveur. Cette requête atterrit sur l'**API** Frappe, qui identifie la fonction Python exacte que j'ai marquée comme "accessible" (via `@frappe.whitelist`).
3. **Logique Métier (Python)** : C'est là que le travail se fait. La fonction **Python (Backend)** commence toujours par une vérification de sécurité (grâce à `_require_admin()`). Ensuite, elle exécute l'action demandée : soit elle récupère des données (filtrer toutes les candidatures de l'année sélectionnée), soit elle les modifie (sauver un nouveau sujet).
4. **Base de Données** : Le Python interagit avec mes DocTypes dans la base de données **MariaDB**.
5. **Mise à Jour** : Le résultat de l'opération (une liste de candidatures filtrée, une confirmation, ou le fichier CSV généré en mémoire) est renvoyé à l'utilisateur via le JavaScript.

5.7 Implémentation des Fonctionnalités Clés

L'implémentation a été la phase la plus intensive, se concentrant sur le développement des interfaces utilisateur (Front-End) et la logique métier (Backend) pour gérer le cycle de vie des mémoires, en ayant choisi comme langue d'affichage l'anglais. Cette phase a consisté à créer un ensemble de routes web spécifiques pour chaque rôle utilisateur.

Architecture du Code (Pages et Controllers)

J'ai structuré mon code autour de **routes web custom** (pages .html) et leurs contrôleurs Python associés (.py), visibles dans la structure de mon application (apps/thesis_application/www/).

L'arborescence détaillée du projet thesis_application est présentée en Annexe A.4

5.7.1. L'Administration

La page /administration (administration.html / administration.py) est le centre de contrôle macro.

Elle permet principalement :

- d'afficher une vue d'ensemble des sujets et des flux liés au système ;
- de gérer l'année académique actuelle (année « en cours ») ;
- de filtrer les données en fonction de l'année académique sélectionnée ;
- masquer les sujets et modifier les informations de candidature.

Un point central de cette page est la **fonction d'Export CSV des candidatures**.

Depuis /administration, l'administration peut générer un fichier CSV contenant **toutes les candidatures de l'année académique sélectionnée**, quel que soit leur statut (Approved, Reject, On Hold).

Le CSV est généré via un appel à une fonction d'API Python et inclut notamment :

- le nom et le prénom de l'étudiant ;
- son e-mail institutionnel ;
- son numéro d'étudiant ;
- la date de candidature ;

Nargice BOUDLAL - Rapport de stage - ESEO

- la date de décision (acceptation / refus), lorsqu'elle existe ;
- le titre du sujet renseigné par l'étudiant ;
- le mail du superviseur ;
- le **statut final** de la candidature (Approved, Reject, On Hold).

Thesis topics publication board

Academic year: 2025-2026 [Export CSV](#)

Academic year in use
Students and professors will only see topics for this academic year. Update it here when a new academic year starts.

2025-2026 [Save](#)

[Filter](#) Search by title, department or professor...

#	Title	Department	Supervisor	Visibility	Action
1	dialyseur	Biomedical	contact.nargice@gmail.c...	Visible	View details
2	test3	Biomedical	contact.nargice@gmail.c...	Hidden	View details
3	test3	Biomedical	contact.nargice@gmail.c...	Hidden	View details
4	Modeling and Control of a Brushless DC ...	Electro-Mechanics	contact.nargice@gmail.c...	Visible	View details

Figure 5 - Espace administration

Supervision détaillée (/details)

La page /details (details.html / details.py) fournit une vue complète d'un sujet de mémoire. Elle affiche :

- un bloc « **Topic information** » avec le département, l'e-mail du superviseur, la description et le **statut sur le portail** (VISIBLE/HIDDEN), ainsi qu'un bouton permettant de **masquer ou afficher le sujet** côté étudiant/professeur ;
- un tableau « **Applications** » listant toutes les candidatures liées au sujet (nom, e-mail, numéro d'étudiant, faculté ULB/VUB, Erasmus, statut, date de candidature et de décision) ;
- un tableau « **Thesis submissions** » récapitulant les mémoires déposés (étudiant, faculté, titre, fichier, date de dépôt), avec une action *Modify* qui ouvre une fenêtre

modale pour **supprimer le PDF** ou **corriger les métadonnées** via l'API `update_submission`.

← Back to administration

dialyseur [show topic on portal](#)

Topic information

Department Biomedical	Supervisor email contact.nargice@gmail.com	Status on portal HIDDEN
Description bdd		

Applications

Last name	First name	Email	Student number	Faculty	Erasmus	Status	Application date	Decision date	Action
jhj	Etudiant	alexandre.borreda@ulb.be	2189	VUB	-	REJECT	2025-12-11 12:55:31.588161	-	-
re	Etudiant	alexandre.borreda@ulb.be	4545	ULB	-	REJECT	2025-12-11 12:58:34.640381	-	-
jhgj	Etudiant	alexandre.borreda@ulb.be	5445	VUB	-	REJECT	2025-12-11 13:05:25.638471	-	-
hhujh	Etudiant	alexandre.borreda@ulb.be	534521	ULB	Yes	APPROVED	2025-12-11 13:15:51.010650	2025-12-11 13:52:03.819012	-

Thesis submissions

Student name	Email	Student number	Faculty	Erasmus	Thesis title	File	Submission date	Action
Etudiant hhujh	alexandre.borreda@ulb.be	534521	ULB	Yes	dialyseur	Download	2025-12-11 13:52:03.819012	Modify

Figure 6 - Supervision détaillée

5.7.2. Le Flux Étudiant

Le parcours de l'étudiant est centré sur la recherche d'un sujet, la candidature et le dépôt final de son mémoire.

Découverte (Page d'accueil)

Lorsqu'il se connecte, l'étudiant est dirigé vers la page `/student` (`student.html` / `student.py`).

Cette page sert de tableau de bord principal et affiche immédiatement la liste des sujets (Thesis Topic) disponibles, sans nécessiter de navigation supplémentaire.

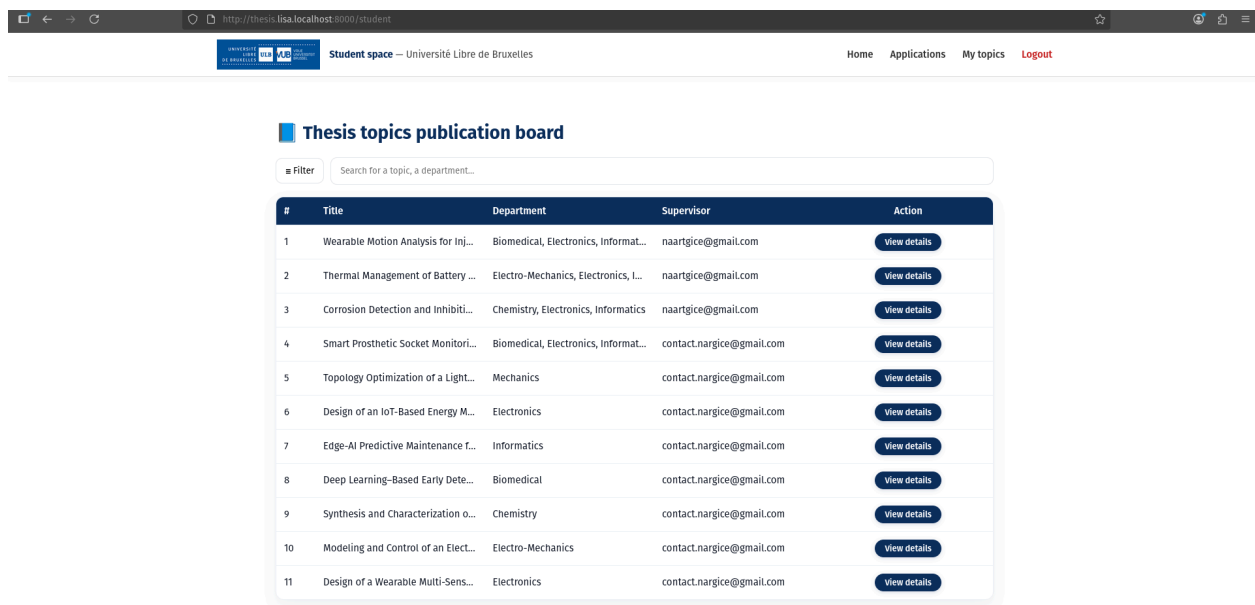


Figure 7 - Espace Étudiant

Consultation d'un sujet

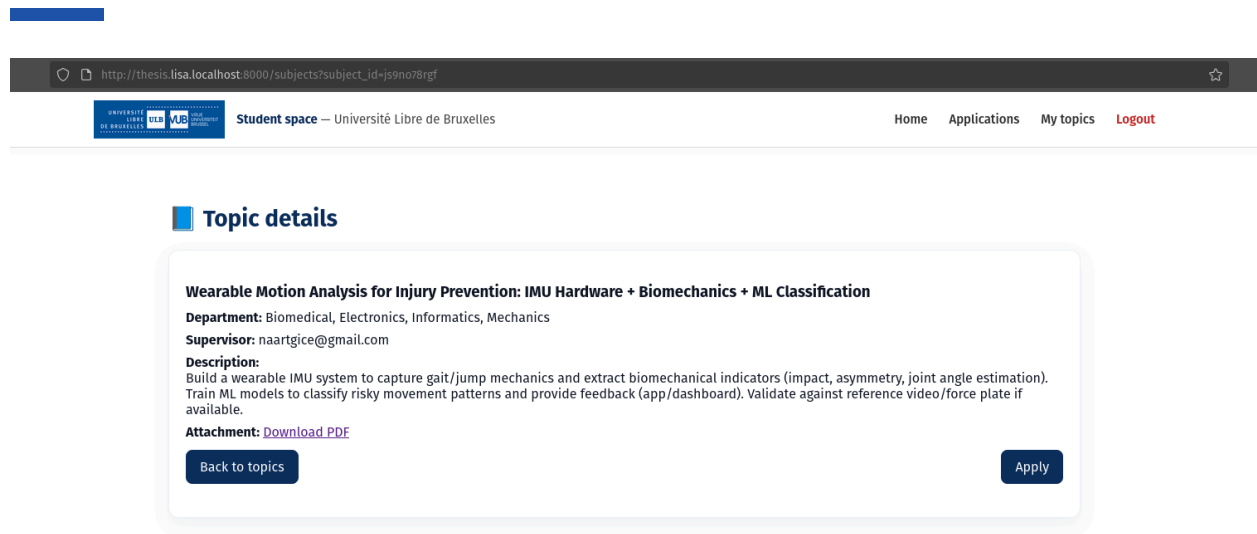
En cliquant sur un sujet, l'étudiant accède à la page /subjects (subjects.html / subjects.py), qui affiche la fiche détaillée du sujet.

S'il souhaite postuler, il est dirigé vers la page /apply (apply.html / [apply.py](#)) en cliquant sur le bouton *apply*

Cette page affiche le formulaire de candidature, dont la soumission est gérée par une fonction d'API qui crée l'enregistrement Thesis Application dans la base de données.

Dans ce formulaire, l'étudiant :

- sélectionne la session d'examen visée ;
- indique s'il prévoit un Erasmus ;
- peut proposer un nouveau titre pour le sujet ;
- **et doit préciser s'il est issu de l'ULB ou de la VUB** (information obligatoire pour le suivi institutionnel).



http://thesis.lisa.localhost:8000/subjects?subject_id=js9no78rgf

UNIVERSITÉ LIBRE DE BRUXELLES Student space — Université Libre de Bruxelles Home Applications My topics Logout

Topic details

Wearable Motion Analysis for Injury Prevention: IMU Hardware + Biomechanics + ML Classification

Department: Biomedical, Electronics, Informatics, Mechanics

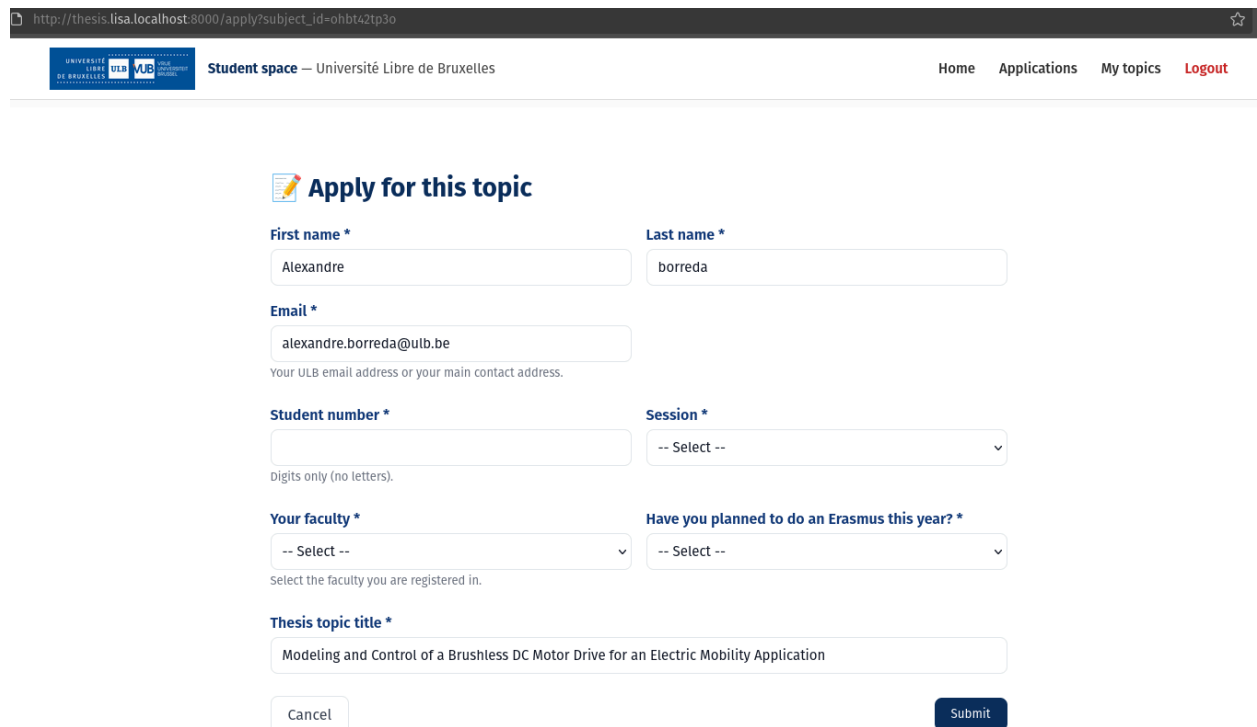
Supervisor: naartgice@gmail.com

Description:
Build a wearable IMU system to capture gait/jump mechanics and extract biomechanical indicators (impact, asymmetry, joint angle estimation). Train ML models to classify risky movement patterns and provide feedback (app/dashboard). Validate against reference video/force plate if available.

Attachment: [Download PDF](#)

[Back to topics](#) [Apply](#)

Figure 8 - Détails d'un sujet de mémoire



http://thesis.lisa.localhost:8000/apply?subject_id=ohbt42tp3o

UNIVERSITÉ LIBRE DE BRUXELLES Student space — Université Libre de Bruxelles Home Applications My topics Logout

Apply for this topic

First name * **Last name ***

Email *
Your ULB email address or your main contact address.

Student number * **Session ***
Digits only (no letters).

Your faculty * **Have you planned to do an Erasmus this year? ***
Select the faculty you are registered in.

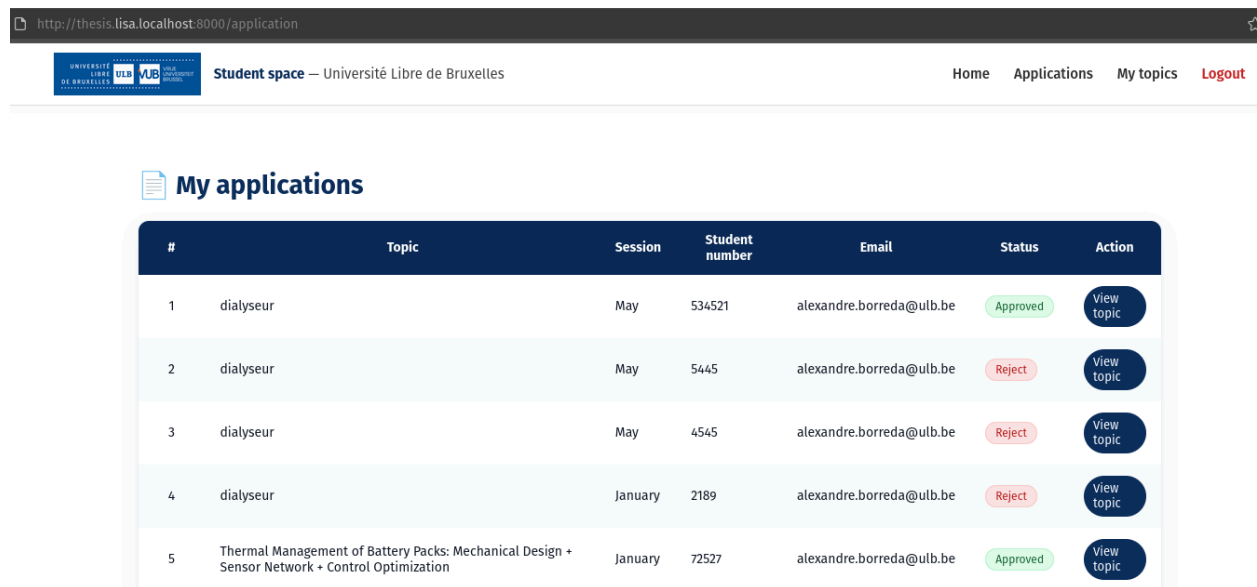
Thesis topic title *

[Cancel](#) [Submit](#)

Figure 9 - Formulaire de candidature

Suivi

L'étudiant peut consulter l'état de ses candidatures via la page /myapply (myapply.html / myapply.py), qui liste toutes ses Thesis Application ainsi que leur statut actuel (Approved, Reject, On Hold.).



#	Topic	Session	Student number	Email	Status	Action
1	dialyseur	May	534521	alexandre.borreda@ulb.be	Approved	view topic
2	dialyseur	May	5445	alexandre.borreda@ulb.be	Reject	view topic
3	dialyseur	May	4545	alexandre.borreda@ulb.be	Reject	view topic
4	dialyseur	January	2189	alexandre.borreda@ulb.be	Reject	view topic
5	Thermal Management of Battery Packs: Mechanical Design + Sensor Network + Control Optimization	January	72527	alexandre.borreda@ulb.be	Approved	view topic

Figure 10 - Page de suivi des candidatures

Dépôt final

Une fois la candidature acceptée, l'étudiant utilise la page /assignment (assignment.html / assignment.py) pour le dépôt du document final.

Cette page vérifie si le champ final_pdf de son enregistrement de candidature est vide et lui propose soit :

- le formulaire d'upload (si aucun PDF n'a encore été soumis) ;
- soit le lien de téléchargement du mémoire déjà déposé.

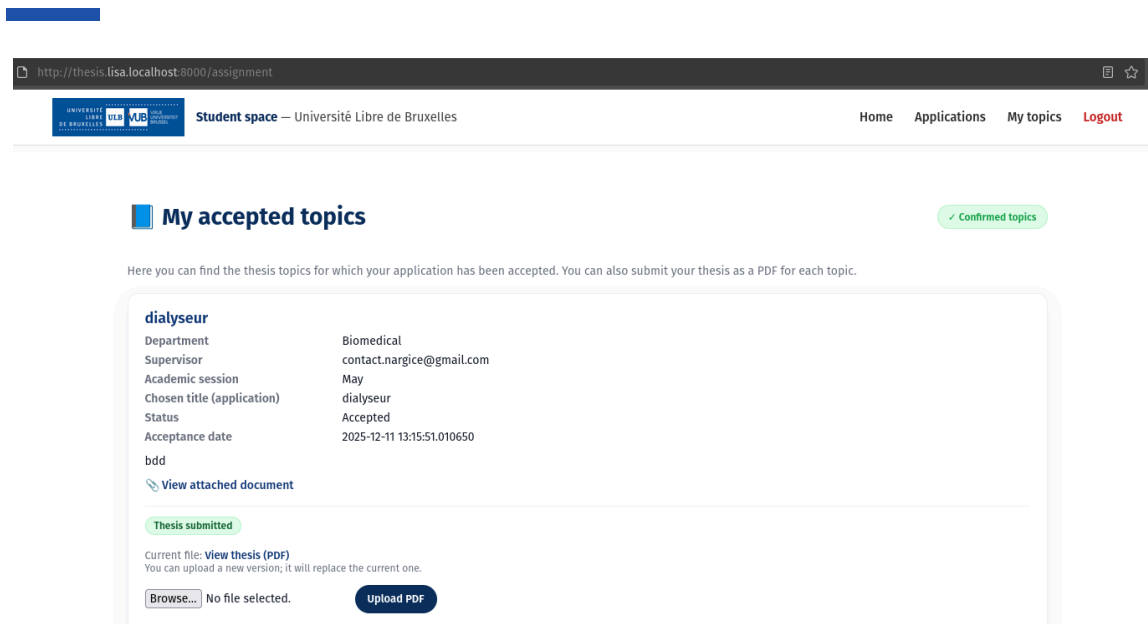


Figure 11 - Page de dépôt de mémoire

5.7.3. Le Flux Professeur

Le professeur utilise différentes pages pour gérer ses sujets et évaluer les étudiants, distinguant la vue globale de ses responsabilités directes.

Vue d'ensemble (/professor)

La page /professor (professor.html / professor.py) sert de page d'accueil pour le professeur. Elle affiche la liste de tous les sujets actifs (Thesis Topic) et lui permet de suivre l'offre globale de l'établissement.

Thesis topics publication board

Filter Search for a topic, a department...

#	Title	Department	Supervisor	Action
1	Wearable Motion Analysis for Inj...	Biomedical, Electronics, Informat...	naartgice@gmail.com	View details
2	Thermal Management of Battery ...	Electro-Mechanics, Electronics, I...	naartgice@gmail.com	View details
3	Corrosion Detection and Inhibiti...	Chemistry, Electronics, Informatics	naartgice@gmail.com	View details
4	Smart Prosthetic Socket Monitori...	Biomedical, Electronics, Informat...	contact.nargice@gmail.com	View details
5	Topology Optimization of a Light...	Mechanics	contact.nargice@gmail.com	View details
6	Design of an IoT-Based Energy M...	Electronics	contact.nargice@gmail.com	View details
7	Edge-AI Predictive Maintenance f...	Informatics	contact.nargice@gmail.com	View details
8	Deep Learning-Based Early Dete...	Biomedical	contact.nargice@gmail.com	View details

Figure 12 - Espace professor

Gestion des propres sujets (/mysubjects)

Pour gérer ses responsabilités, le professeur utilise la page /mysubjects (mysubjects.html / mysubjects.py).

Cette page filtre la liste des sujets pour n'afficher que ceux où il est désigné comme superviseur.

Elle offre également un **filtre par année académique**, ce qui permet au professeur de :

- consulter ses sujets pour l'année en cours ;
- naviguer dans les années précédentes pour retrouver ses anciens sujets.

Le professeur dispose d'une vue d'ensemble des sujets de mémoire approuvés (Approved) et de ceux en attente de traitement (On Hold), et peut consulter directement le PDF du mémoire soumis par l'étudiant lorsque celui-ci est disponible.

My thesis topics Academic year: 2025-2026 Export subjects (csv)

Search for a topic, a department...

#	Title	Department	Supervisor	Status	PDF	Action
1	dialyseur	Biomedical	contact.nargice@g...	Approved	Open PDF	View details
2	test3	Biomedical	contact.nargice@g...	Approved	-	View details
3	Modeling and Contr...	Electro-Mechanics	contact.nargice@g...	-	-	View details

Figure 13 - Gestion/Archivage des sujets du professeur

Création et suivi des candidatures

Le professeur crée de nouveaux sujets via la page /publish (publish.html / publish.py).

Publish a thesis topic

Title *

Description *

Department *

☐ Biomedical ☐ Chemistry ☐ Electro-Mechanics
☐ Electronics ☐ Informatics ☐ Mechanics

Confidential

No

Attachment (optional)

BROWSE... No file selected.

PDF format only. Maximum file size: 10 MB.

Publish Cancel

Figure 14 - Page de publication de sujet de mémoire

Historique d'un sujet (/history)

La page /history (history.html / history.py) est accessible depuis /mysubjects via le bouton « **View details** » pour un sujet donné.

Elle affiche :

- un bloc « **Topic details** » avec le titre du sujet, le département, l'e-mail du superviseur, la description, un éventuel badge *Confidential topic* et, le cas échéant, un lien **Download PDF** pour la pièce jointe du sujet ;
- un tableau « **Applications for this topic** » listant toutes les candidatures liées à ce sujet, avec pour chaque étudiant :
 - le nom (Student) ;
 - le numéro d'étudiant (Number) ;
 - la session choisie ;
 - le titre proposé ;
 - le statut de la candidature (Approved / Reject /On Hold) ;
 - l'état du mémoire (pas encore soumis, ou lien **Download PDF** lorsque le fichier est disponible).

The screenshot shows a web interface for a professor's space. The top navigation bar includes links for Home, Publish, Applications, My topics, and Logout. The main content area is titled 'Topic details' and features a 'Confidential topic' badge. The topic title is 'Smart Prosthetic Socket Monitoring: Pressure + Temperature Sensing with AI-Based Comfort Prediction'. Below the title, it lists the Department (Biomedical, Electronics, Informatics), Supervisor (contact.nargice@gmail.com), and a detailed Description. An 'Attachment: Download PDF' link is provided. At the bottom, there is a table titled 'Applications for this topic' with columns for Student, Number, Session, Proposed title, Status, and Thesis. One application is listed for 'alexandre borreda' with status 'Approved' and a 'Download PDF' link.

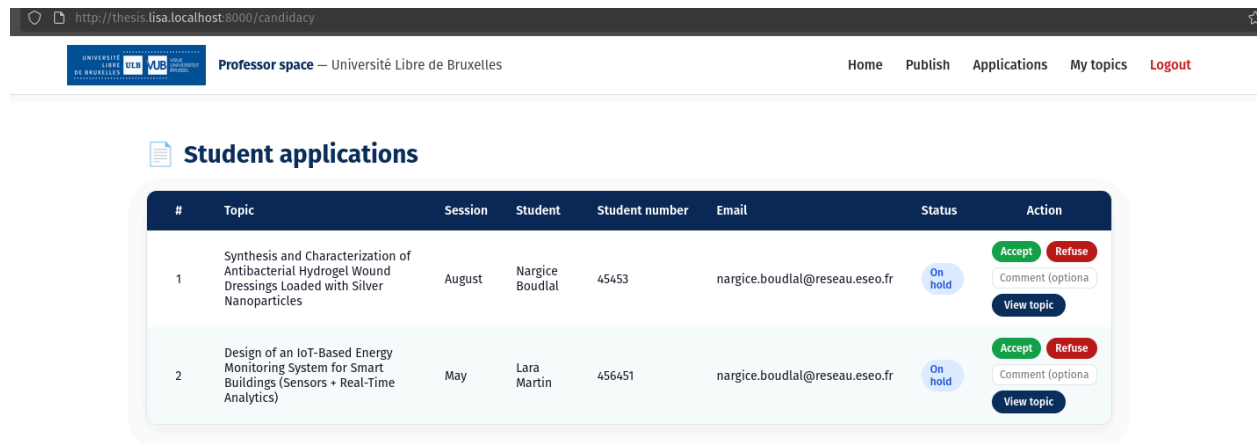
Student	Number	Session	Proposed title	Status	Thesis
alexandre borreda	p241105	May	Smart Prosthetic Socket Monitoring: Pressure + Temperature Sensing with AI-Based Comfort Prediction + test	Approved	Download PDF

Figure 15 - Page de gestion de sujet par le professeur

Gestion des candidatures (/candidacy)

Le professeur accepte ou refuse une candidature via la page /candidacy.

Lors d'un refus, il a la possibilité de laisser un commentaire explicatif à destination de l'étudiant, afin de justifier sa décision.



#	Topic	Session	Student	Student number	Email	Status	Action
1	Synthesis and Characterization of Antibacterial Hydrogel Wound Dressings Loaded with Silver Nanoparticles	August	Nargice Boudlal	45453	nargice.boudlal@reseau.eseo.fr	On hold	Accept Refuse Comment (optional) View topic
2	Design of an IoT-Based Energy Monitoring System for Smart Buildings (Sensors + Real-Time Analytics)	May	Lara Martin	456451	nargice.boudlal@reseau.eseo.fr	On hold	Accept Refuse Comment (optional) View topic

Figure 16 - Gestion des candidatures

5.7.4. Gestion des années académiques – [utils.py](#)

Le fichier utils.py centralise la gestion des années académiques à partir du Doctype de configuration Thesis Settings.

- **get_current_academic_year(default=None)**
Renvoie l'année académique courante définie dans Thesis Settings (par ex. 2024-2025). Si rien n'est configuré ou en cas d'erreur, la fonction renvoie la valeur default.
- **get_all_academic_years()**
Lit le champ texte known_academic_years dans Thesis Settings (une année par ligne ou séparée par des virgules), le nettoie et renvoie la liste triée de toutes les années connues, en s'assurant que l'année courante y figure.

Ces fonctions sont réutilisées dans les différentes pages Python (par exemple /student, /mysubjects, /administration) pour alimenter les filtres et listes déroulantes d'années académiques de manière cohérente.

5.7.5. Logique serveur [api.py](#) et [hooks.py](#)

Logique métier et notifications – api.py

Le fichier api.py contient la principale logique serveur de l'application :

- Gestion des sujets
 - create_thesis_topic(...) : création d'un *Thesis Topic* pour le professeur connecté.
 - get_professor_topics() : récupère les sujets du professeur.
 - get_thesis_for_student() : renvoie les sujets visibles pour l'étudiant.
- Exports et visibilité
 - export_accepted_students(subject_id) : export CSV des candidatures acceptées pour un sujet.
 - download_applications_csv() : export CSV de toutes les candidatures, réservé au rôle Administration.
 - set_subject_visibility(subject_name, hidden) : permet à Administration de masquer/afficher un sujet sur le portail étudiant.
- Mails automatiques liés aux candidatures
 - notify_professor_new_application(doc) : envoie un e-mail au professeur lorsqu'un étudiant candidate sur son sujet.
 - notify_student_decision(doc) : envoie un e-mail à l'étudiant quand sa candidature est acceptée ou refusée.
 - on_thesis_application_insert(...) et on_thesis_application_update(...) sont les hooks qui déclenchent ces envois au moment de la création ou de la mise à jour d'une *Thesis Application*.

Intégration Frappe et hooks – hooks.py

Le fichier `hooks.py` déclare comment l'application s'intègre dans Frappe :

- Page d'accueil et redirection par rôle
 - `home_page = "login"` : les invités arrivent sur la page de login.
 - `role_home_page` : redirige automatiquement l'utilisateur après connexion vers `/student`, `/professor` ou `/administration` selon son rôle.
- Hooks sur le Doctype Thesis Application
 - `doc_events` :
 - `after_insert` → `thesis_application.api.on_thesis_application_insert`
 - `on_update` → `thesis_application.api.on_thesis_application_update`Ces hooks assurent que les e-mails au professeur et à l'étudiant sont envoyés au bon moment.
- Routage des pages web
 - `website_route_rules` mappe des URLs (par ex. `/administration`, `/subjects`, `/details`) vers les pages correspondantes dans le dossier `www` de l'application.

En deux mots : `api.py` contient la logique métier (création, exports, mails), et `hooks.py` “branche” cette logique sur les événements Frappe et les routes web.

5.7.6. GitLab

Dans ce projet, GitLab n'est pas seulement un simple dépôt de code : il sert surtout de source centrale pour le déploiement de l'application dans un environnement Docker.

Le code de l'application `thesis_application` est hébergé dans le projet GitLab ULB :

- Groupe : LISA / MFE-EPB
- Projet : THESIS MANAGEMENT
- Dépôt : `thesis-management`
- Branche de travail : `develop`

La synchronisation avec GitLab se fait principalement via le terminal. Lorsqu'une évolution est prête, j'utilise systématiquement la séquence de commandes suivante :

Nargice BOUDLAL - Rapport de stage - ESEO

- **git status** permet de voir les fichiers modifiés.
- **git add -A** ajoute toutes les modifications (ajouts, suppressions, mises à jour).
- **git commit -m "chore: initial push of thesis_application"**
- enregistre ces changements avec un message décrivant la modification.
- **git pull --rebase origin develop** met à jour ma branche locale en rejouant mes commits au-dessus de la dernière version distante, ce qui garde un historique plus linéaire.
- **git push -u origin develop** envoie finalement le code sur la branche develop du dépôt GitLab ULB.

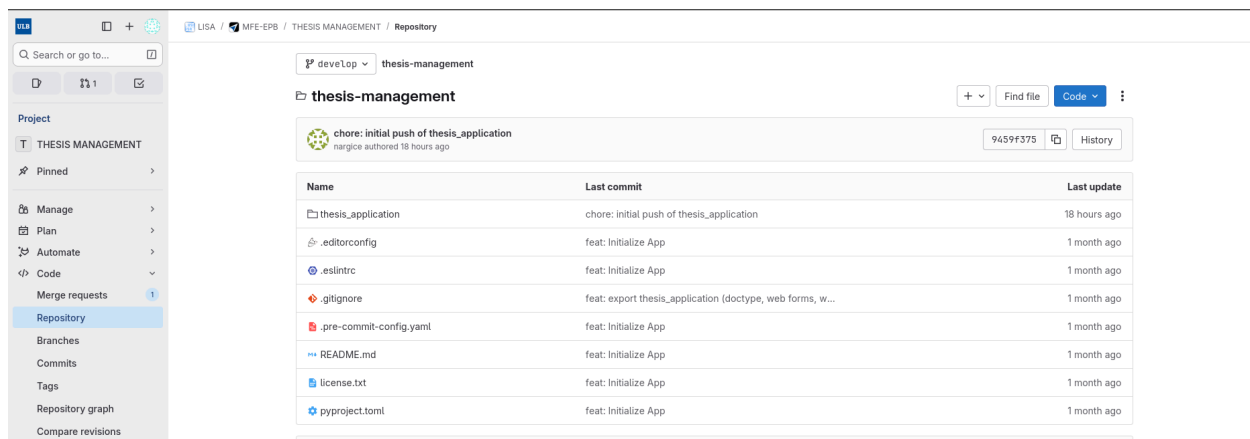


Figure 17 - Gitlab du projet

Après avoir poussé mon code sur GitLab, le dépôt thesis-management devient la source utilisée pour déployer l'application dans des conteneurs Docker sur le serveur mais aussi un moyen pour mes tuteurs de suivre l'avancé du projet.

6. Mise en place de l'envoi des notifications par e-mail

Pour envoyer automatiquement des e-mails aux professeurs et aux étudiants, j'ai fait deux choses

- Configurer un compte e-mail sortant dans Frappe
- Appeler `frappe.sendmail` au bon moment dans le code de l'application

6.1. Configuration du compte e-mail

Dans Frappe, j'ai créé un **Email Account** nommé LISA :

- Adresse : lisa.ulb.ac.be@gmail.com
- Service : **GMail**
- Méthode d'authentification : *De base* (mot de passe / mot de passe d'application Gmail)
- SMTP sortant :
 - Serveur : smtp.gmail.com
 - Port : 587
 - TLS activé
 - « Activer sortant » + « Sortant par défaut » cochés

J'ai également ajouté une signature au compte et vérifié la configuration via l'envoi d'un e-mail de test.

The screenshot shows the 'Compte Email' configuration page for an account named 'LISA'. The page is divided into several sections:

- Compte**:
 - Adresse électronique: lisa.ulb.ac.be@gmail.com
 - Service: GMail
- Authentification**:
 - Méthode: De base
 - Mot de Passe: [masked]
 - ☐ En attente Mot de Passe
 - ☐ Utiliser le codage ASCII pour le mot de passe
- Configuration POP/IMAP**:
 - ☐ Activer Entrant
- Lien vers les documents**:
 - ☒ Créer des contacts à partir des e-mails entrants
 - ☐ Notifier si aucune réponse
 - ☐ Activer la liaison automatique dans les documents
- Outgoing (SMTP) Settings**:
 - ☒ Activer Sortant
 - ☒ Sortant par Défaut
 - ☒ Utiliser TLS
 - ☒ Toujours utiliser cet email comme expéditeur

On the left sidebar, there are options for 'Assigné À', 'Pièces jointes', 'Balises', and 'Partager', each with a plus icon. Below these is a 'SUIVRE' button and a message: 'Vous avez créé ceci - Il ya 1 semaine'.

Figure 18 - Mise en place du protocole SMTP

6.2. Intégration dans l'application

Côté application, j'utilise l'API intégrée de Frappe :

- Lorsqu'un **étudiant soumet une candidature** (/apply) :
après avoir enregistré la candidature, j'appelle une fonction Python qui envoie un e-mail au professeur (via `frappe.sendmail`) pour lui signaler qu'un étudiant a postulé et l'inviter à se connecter à son espace.
- Lorsqu'un **professeur change le statut** d'une candidature (Approved / Reject) :
après mise à jour du champ status, j'appelle une autre fonction qui envoie un e-mail à l'étudiant afin de lui communiquer la décision et de l'inviter à consulter son espace.

Ces fonctions de notification sont regroupées dans un module utilitaire de l'application, ce qui permet de centraliser tout le contenu des messages et les appels à `frappe.sendmail`.

Un extrait de la fonction de notification (professeur) et son déclenchement via `hooks.py` est donné en Annexe B

7. Déploiement Docker

Pour mettre en production mon application `thesis_application`, j'ai mis en place un environnement Frappe basé sur Docker, en suivant la documentation officielle du dépôt GitHub `frappe_docker`. L'application est ensuite déployée à partir du dépôt GitLab de l'ULB :

Dépôt GitLab : <https://gitlab.ulb.be/lisa/mfe-epb/thesis-management.git>

L'objectif est d'héberger le portail de gestion des sujets de mémoire dans un environnement isolé, reproductible et facile à maintenir.

Le script complet de déploiement Docker utilisé est disponible en Annexe A.2.

7.1. Installation de l'environnement Frappe Docker

1. Récupération de la configuration Docker officielle :

Nargice BOUDLAL - Rapport de stage - ESEO

git clone https://github.com/frappe/frappe_docker.git
cd frappe_docker

2. Configuration de l'environnement en suivant la documentation `frappe_docker` :
 - définition des variables d'environnement (configuration Frappe, base de données, utilisateur administrateur, etc.) ;
 - préparation du fichier `pwd.yml` qui décrit les services nécessaires (backend Frappe, base de données, etc.) ;
 - configuration des volumes pour la persistance des données (base de données, fichiers du site, logs).
3. Démarrage de l'environnement :

docker compose -f pwd.yml up -d

À ce stade, l'instance Frappe tourne dans des containers Docker décrits par `pwd.yml`.

7.2. Création du site `thesis.lisa.ulb.ac.be`

Une fois l'environnement Frappe opérationnel, j'ai créé le site Frappe qui héberge l'application.

1. Connexion au container backend :

cd ~/frappe_docker
docker compose -f pwd.yml exec backend bash

2. Accès au bench Frappe :

cd /home/frappe/frappe-bench

3. Création du site `thesis.lisa.ulb.ac.be` :

bench new-site thesis.lisa.ulb.ac.be

Pendant cette étape, Frappe crée la base de données du site et me demande de définir le mot de passe administrateur.

C'est ce site (thesis.lisa.ulb.ac.be) qui sert ensuite de base à tout le portail de gestion des sujets de mémoire.

7.3. Déploiement de l'application thesis_application depuis GitLab

Après la création du site, j'ai déployé mon application personnalisée thesis_application sur ce site, en utilisant le dépôt GitLab de l'ULB.

1. Récupération de l'application depuis GitLab :

bench get-app thesis_application

<https://gitlab.ulb.be/lisa/mfe-epb/thesis-management.git>

2. Installation des dépendances Python :

bench setup requirements --python

3. Installation de l'application sur le site thesis.lisa.ulb.ac.be :

bench --site thesis.lisa.ulb.ac.be install-app thesis_application

bench --site thesis.lisa.ulb.ac.be migrate

bench --site thesis.lisa.ulb.ac.be clear-cache

- install-app installe l'application sur le site que j'ai créé ;
- migrate applique les migrations de base de données ;
- clear-cache force Frappe à recharger la configuration et les nouveaux éléments (Doctypes, pages web, scripts, etc.).

4. Vérification de l'import du module Python :

./env/bin/python - << 'PY'

import thesis_application

print("OK, import thesis_application depuis :", getattr(thesis_application, "file",

"))

PY

7.4. Redémarrage du backend après déploiement

Après le déploiement de l'application et l'exécution des migrations, j'ai redémarré le service backend pour prendre en compte les changements :

```
cd ~/frappe_docker
```

```
docker compose -f pwd.yml restart backend
```

```
docker compose -f pwd.yml logs backend --tail=50
```

La commande logs me permet de vérifier que le backend redémarre correctement et que le site thesis.lisa.ulb.ac.be fonctionne avec la version à jour de l'application.

IMPACT DU PROJET

I. Impact écologique

Sur le plan écologique, la solution reste un service numérique qui consomme des ressources (serveur, base de données, conteneurs Docker qui tournent en continu). Je suis consciente que l'utilisation d'un ERP complet comme ERPNext/Frappe n'est pas neutre d'un point de vue énergétique.

En revanche, le projet apporte plusieurs améliorations :

- la gestion des sujets et des candidatures est **centralisée** dans un seul outil, au lieu de reposer sur un ensemble de scripts "bricolés" et de fichiers dispersés ;
- cela limite la multiplication de tableaux recréés chaque année, de pièces jointes envoyées à répétition et de versions différentes qui circulent ;
- la solution est pensée pour être **réutilisée** d'une année à l'autre, ce qui évite de "refaire" un système à chaque session de mémoires.

L'enjeu, pour la suite, sera d'optimiser l'hébergement (dimensionnement des conteneurs, mutualisation des ressources) afin de réduire l'empreinte tout en gardant un outil fiable.

Nargice BOUDLAL - Rapport de stage - ESEO

II. Impact social

Sur le plan social, l'impact le plus important concerne les **étudiants**, qu'ils soient francophones ou néerlandophones. L'outil que j'ai développé propose une interface unique, en **anglais**, mais commune à tous. Cela signifie :

- un **même portail** pour consulter les sujets de mémoire et s'inscrire, quel que soit le cursus linguistique
- un **processus homogène** pour tous les étudiants, avec les mêmes étapes et les mêmes règles
- moins de confusion liée à des formulaires différents ou à des procédures séparées selon les groupes.

CONCLUSION

L'objectif de ce projet était de remplacer un dispositif historique devenu fragile (LimeSurvey + scripts Python) par une solution **open source**, **robuste** et **maintenable**, capable de gérer un processus unique de mémoires pour des étudiants **ULB et VUB**. Le choix d'**ERPNext/Frappe** a fourni un socle structurant (Doctypes, rôles/permissions, API, hooks) garantissant la **traçabilité** des candidatures et une **séparation claire des accès** selon les profils.

Le module **thesis_application** mis en place couvre un **workflow complet** : publication des sujets, candidatures, décision du professeur, suivi côté étudiant, dépôt du PDF final et supervision par l'administration. Les principaux résultats sont la mise à disposition de **trois espaces dédiés**, la **redirection automatique après connexion**, le **filtrage par année académique**, la **gestion de la visibilité des sujets**, l'**identification ULB/VUB**, l'**export CSV** des candidatures par année, ainsi que l'**automatisation des notifications e-mail** lors d'une candidature et d'une décision, réduisant les échanges informels et améliorant la continuité du suivi.

Sur le plan personnel, j'ai beaucoup apprécié ce stage : travailler dans un environnement **Linux** et apprendre à utiliser **Docker** m'a particulièrement plu, et cette expérience renforce mon envie d'évoluer vers des domaines liés à l'**infrastructure** et aux **réseaux**. J'ai également eu la chance d'être très bien entourée et accompagnée tout au long du projet ; je remercie une nouvelle fois

Nargice BOUDLAL - Rapport de stage - ESEO



toute l'équipe du **LISA** pour son accueil et sa disponibilité. Enfin, je remercie aussi mon camarade **Alexandre Borreda**, également en stage au LISA, avec qui j'ai pu échanger et progresser tout au long de cette expérience.

ANNEXES

Annexe A — Installation & Déploiement

A.1 – Script d’installation de l’environnement local (Bench)

Cette annexe donne le script complet utilisé pour mettre en place l’environnement de développement local Frappe/ERPNext (décrit de façon plus pédagogique dans la section 4.1 et 4.2 du rapport).

Création d'un environnement virtuel Python

```
python3 -m venv env
```

```
source env/bin/activate
```

Installation de bench

```
pip install frappe-bench
```

Initialisation du bench (dossier de travail)

```
bench init thesis-bench
```

```
cd thesis-bench
```

Création du site local

```
bench new-site thesis.lisa.localhost
```

Création de l'application

```
bench new-app thesis_application
```

Installation de l'application sur le site

```
bench --site thesis.lisa.localhost install-app thesis_application
```

Nargice BOUDLAL - Rapport de stage - ESEO

Lancement du serveur de développement

bench start

A.2 – Structure type d’une application Frappe

Cette annexe complète la description de la structure d’une application Frappe, évoquée dans la section 5.5 du rapport.

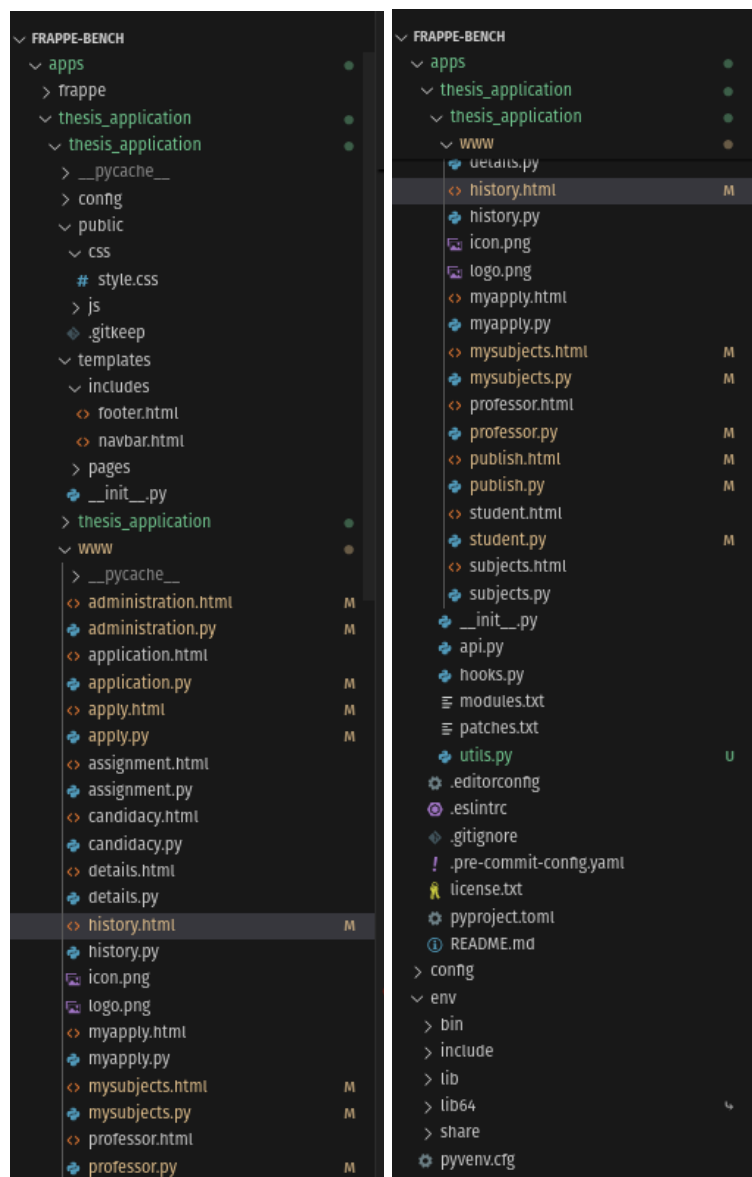
```
apps/library_management
├─ README.md
├─ library_management
│   └─ hooks.py
│   └─ library_management
│       └─ __init__.py
│   └─ modules.txt
│   └─ patches.txt
│   └─ public
│       └─ css
│       └─ js
│   └─ templates
│       └─ __init__.py
│       └─ includes
│       └─ pages
│       └─ __init__.py
│   └─ www
└─ pyproject.toml
```

Structure d’une application Frappe

Cette arborescence illustre les dossiers générés automatiquement par **bench new-app** et le positionnement des pages web / contrôleurs utilisés dans le projet.

A.3 – Arborescence du projet thesis_application

Cette annexe détaille, plus précisément, les parties utilisées dans le projet (décrites dans la section « Architecture du Code (Pages et Controllers) »).



Structure du dossier thesis_application

A.4 – Script de déploiement Docker

Cette annexe regroupe le **script de déploiement complet** de l'application **thesis_application** sur l'environnement Frappe Docker, présenté de manière plus narrative dans la section 5 du rapport.

1. Récupération de la configuration Frappe Docker

Nargice BOUDLAL - Rapport de stage - ESEO

```
git clone https://github.com/frappe/frappe_docker.git
```

```
cd frappe_docker
```

2. Démarrage de l'environnement décrit dans pwd.yml

```
docker compose -f pwd.yml up -d
```

3. Connexion au conteneur backend

```
docker compose -f pwd.yml exec backend bash
```

4. Passage dans le bench du conteneur

```
cd /home/frappe/frappe-bench
```

5. Création du site de production

```
bench new-site thesis.lisa.ulb.ac.be
```

6. Récupération de l'application depuis GitLab

```
bench get-app thesis_application https://gitlab.ulb.be/lisa/mfe-epb/thesis-management.git
```

```
bench get-app thesis_application \
```

```
https://gitlab.ulb.be/lisa/mfe-epb/thesis-management.git \
```

```
--branch develop
```

7. Installation des dépendances Python

```
bench setup requirements --python
```

Nargice BOUDLAL - Rapport de stage - ESEO



8. Installation de l'application sur le site

```
bench --site thesis.lisa.ulb.ac.be install-app thesis_application
```

```
bench --site thesis.lisa.ulb.ac.be migrate
```

```
bench --site thesis.lisa.ulb.ac.be clear-cache
```

9. Sortie du conteneur et redémarrage du backend

```
exit
```

```
docker compose -f pwd.yml restart backend
```

```
docker compose -f pwd.yml logs backend --tail=50
```

Annexe B — Notification par mail

```
def notify_professor_new_application(doc):
    """Prévenir le professeur quand un étudiant candidate à son sujet."""

    # On suppose que Thesis Application a un champ "thesis" (Link vers Thesis Topic)
    if not getattr(doc, "thesis", None):
        return

    # Récupérer l'email du prof depuis le sujet
    thesis_title, professor = frappe.db.get_value(
        "Thesis Topic",
        doc.thesis,
        ["title", "professor"],
    ) or (doc.title, None)

    # Si pas de prof → on ne fait rien
    if not professor:
        return

    subject = f"Nouvelle candidature pour votre sujet : {thesis_title or doc.title}"
    message = f"""
    Bonjour,

    Un étudiant vient de candidater à votre sujet de mémoire :

    - Sujet      : {thesis_title or doc.title}
    - Étudiant   : {doc.student_name}
    - Email      : {doc.student_mail}
    - Session    : {doc.session}

    Merci de vous connecter à la plateforme pour traiter cette candidature.
    """.strip()

    frappe.sendmail(
        recipients=[professor], # dans Thesis Topic, le champ "professor" contient normalement l'utilisateur (email) du prof
        subject=subject,
        message=message,
    )
```

*Fonction d'envoi de notification au professeur
(définie dans api.py et déclenchée via hooks.py (doc_events))*

Cet extrait présente la fonction **notify_professor_new_application(doc)** (définie dans api.py) : elle est déclenchée automatiquement lors de la création d'une Thesis Application via le hook `after_insert` déclaré dans hooks.py, et envoie un e-mail au superviseur pour l'informer qu'un étudiant a candidaté et l'inviter à se connecter afin de traiter la demande.

Annexe C — Principales ressources de documentation utilisées

Cette annexe regroupe les sources documentaires qui ont servi de base aux choix techniques.

- **Installation de Bench et de Frappe**

- <https://docs.frappe.io/framework/user/en/installation>

<https://docs.frappe.io/framework/user/en/tutorial/install-and-setup-bench>

Création d'application et de site Frappe

- <https://docs.frappe.io/framework/user/en/tutorial/create-an-app>

<https://docs.frappe.io/framework/user/en/tutorial/create-a-site>

Gestion des comptes e-mail / SMTP

- <https://docs.frappe.io/erpnext/user/manual/en/email-account>

Frappe Docker (déploiement)

- https://github.com/frappe/frappe_docker

Script API et logique serveur

- <https://docs.frappe.io/framework/v14/user/en/desk/scripting/script-api>

<https://docs.frappe.io/framework/user/en/desk/scripting/server-script>

Annexe D — Table des figures et illustrations utilisées dans le rapport

- Figure 1 — Frappe Framework architecture (p. 14)
- Figure 2 — Application “thesis_application” installé (p. 19)
- Figure 3 — Interface de gestion des doctypes (p. 21)
- Figure 4 — Interface de gestion des rôles (p. 21)
- Figure 5 — Espace administration (p. 24)
- Figure 6 — Supervision détaillée (p. 25)
- Figure 7 — Espace Étudiant (p. 26)
- Figure 8 — Détails d’un sujet de mémoire (p. 27)
- Figure 9 — Formulaire de candidature (p. 27)
- Figure 10 — Page de suivi des candidatures (p. 28)
- Figure 11 — Page de dépôt de mémoire (p. 29)
- Figure 12 — Espace professor (p. 30)
- Figure 13 — Gestion/Archivage des sujets du professeur (p. 31)
- Figure 14 — Page de publication de sujet de mémoire (p. 31)
- Figure 15 — Page de gestion de sujet par le professeur (p. 32)
- Figure 16 — Gestion des candidatures (p. 33)
- Figure 17 — Gitlab du projet (p. 36)
- Figure 18 — Mise en place du protocole SMTP (p. 37)

Annexe E — Glossaire des définitions et acronymes utilisés

Acronymes

API : *Application Programming Interface* : fonctions côté serveur exposées et appelées depuis le frontend (ex. via @frappe.whitelist).

CSV : *Comma-Separated Values* : format de fichier texte utilisé pour l’export des données.

ERP : *Enterprise Resource Planning* : progiciel de gestion intégré.

LISA : *Laboratory of Image Synthesis and Analysis* : laboratoire d’accueil (ULB).

SMTP : *Simple Mail Transfer Protocol* : protocole d’envoi d’e-mails utilisé pour les notifications.

TLS : *Transport Layer Security* : chiffrement activé pour l’envoi SMTP (ex. port 587 + TLS).

ULB : Université Libre de Bruxelles.

VUB : Vrije Universiteit Brussel.

Termes

Backend : Partie serveur (Frappe) qui exécute la logique métier et sert l'application.

Bench : Outil en ligne de commande Frappe utilisé pour initialiser l'environnement, créer sites/apps et lancer le serveur de dev (bench start).

Desk : Interface web interne de Frappe/ERPNext utilisée pour la configuration (Doctypes, rôles, permissions), pas pour développer les pages "utilisateur final".

Dev Container : Environnement de développement conteneurisé envisagé puis abandonné (trop complexe à stabiliser).

DocType : Entité métier centrale de Frappe (Type de Document) définie par métadonnées, représentant les données (ex. Thesis Application, Thesis Topic, etc.).

Docker : Technologie de conteneurisation utilisée pour exécuter l'environnement de manière isolée et reproductible.

Docker Compose : Outil d'orchestration multi-conteneurs via un fichier .yaml/docker-compose.yml (services backend/db/redis, volumes, réseau).

ERPNext : ERP open source basé sur le framework Frappe (socle technique retenu).

Frappe : Framework full-stack (Python) sur lequel s'appuie ERPNext, avec gestion intégrée des Doctypes, rôles/permissions, etc.

GitLab : Plateforme de versioning utilisée comme dépôt et source de déploiement (dépôt ULB "thesis-management").

Hook (Frappe) : Mécanisme reliant une fonction (ex. dans api.py) à un événement (ex. after_insert, on_update) pour automatiser des actions (ex. e-mails).

Jinja2 : Moteur de templates côté serveur utilisé pour les pages web (HTML).

MariaDB : Base de données utilisée pour stocker les données des sites/doctypes.

Redis : Service utilisé notamment pour le cache et/ou les files d'attente dans le stack Frappe/ERPNext.

Rôles (Student / Professor / Administration) : Profils d'accès qui déterminent permissions et comportement de l'application.

Site (Frappe) : Instance Frappe configurée (ex. thesis.lisa.ulb.ac.be) sur laquelle l'app est installée via Bench.



VS Code : Éditeur utilisé pour le développement “full code” (pages web + logique Python) en dehors du Desk.